Data Structures & Algorithms: Binary Search Trees (BST)

1. Tree Properties and Search Invariants

A Binary Search Tree enforces the strict invariant that for every node X, all values in its left subtree are strictly smaller than X, and all values in its right subtree are strictly greater.

```
class BSTNode:
    def __init__(self, key: int):
        self.key = key
        self.left = None
        self.right = None

def bst_insert(root: BSTNode, key: int) -> BSTNode:
    if root is None:
        return BSTNode(key)
    if key < root.key:
        root.left = bst_insert(root.left, key)
    elif key > root.key:
        root.right = bst_insert(root.right, key)
    return root

def bst_search(root: BSTNode, target: int) -> bool:
    if root is None:
        return False
    if root.key == target:
        return True
    elif target < root.key:
        return bst_search(root.left, target)
    return bst_search(root.right, target)
```

Time Complexity: Average search time is O(log n) in balanced trees and degrades to O(n) in pathological degenerate cases.